Qawafel BDD Test Project — User Manual

Project Setup & Structure Guide

Version: 1.0
Date: April 21, 2026
Author: QA Team

Table of Contents

1. Overview
2. Prerequisites
3. Project Initialization
4. Folder Structure
5. Configuration Files
6. Writing Feature Files (Gherkin Scenarios)
7. Prompt Templates
8. Execution Workflow
9. Evidence Collection
10. Bug Reporting
11. GitHub Issue Publishing
12. Fixtures & Helper Scripts
13. Complete Prompt Reference

---

1. Overview

This project is a **BDD (Behavior-Driven Development) manual testing framework** powered by:

- **Gherkin `.feature` files** for scenario authoring
- `playwright-cli` for browser-based manual execution with headed Chromium
- **GitHub Copilot Agent** (Claude) as the execution engine
- **GitHub CLI (`gh`)** for bug report publishing

The framework is designed for exploratory and structured manual testing against Qawafel development environments, with full evidence capture (screenshots) and structured bug reporting.

---

2. Prerequisites

Tool	Purpose	Install Command
Node.js (v18+)	Runtime for package management	Download from nodejs.org
VS Code	IDE with Copilot agent	Download from code.visualstudio.com
GitHub Copilot	AI agent for scenario execution	VS Code extension
playwright-cli	Browser automation tool	Bundled via `.claude_skills`
GitHub CLI	Publish issues to GitHub	`winget install GitHub.cli`

---

3. Project Initialization

Step 1: Create the project folder

```
mkdir "GIT_Test Project"
cd "GIT_Test Project"
```

Step 2: Initialize Node.js project

```
npm init -y
```

This creates `package.json` :

```
{
  "name": "git_test-project",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Step 3: Create all required directories

```
mkdir scenarios\Admin
mkdir fixtures
mkdir prompts
mkdir evidence
mkdir bug_reports
```

Step 4: Install Playwright browser

```
playwright-cli install-browser
```

Step 5: Authenticate GitHub CLI

```
gh auth login --hostname github.com --git-protocol https --web
```

---

4. Folder Structure

```
GIT_Test Project/
│
├── package.json                          # Node.js project config
│
├── fixtures/                             # Test data & helper scripts
│   ├── credentials.yml                   # Environment URLs, mobiles, OTPs
│   ├── upload-files.js                   # Playwright file upload helper
│   ├── check-response.js                # API response interceptor
│   ├── expire-otp.js                    # OTP timer expiry simulator
│   └── uploads/                         # Test files for upload scenarios
│       └── commercial-registration.jpg
│
├── scenarios/                            # Gherkin feature files
│   └── Admin/
│       ├── app_login.feature            # Login & vendor store access scenarios
│       └── register.feature             # Registration scenarios
│
├── prompts/                              # Reusable agent prompts
│   ├── Execution.md                     # Scenario execution prompt
│   ├── Bug_Report.md                    # Bug report generation prompt
│   ├── Github_issue.md                  # GitHub issue publishing prompt
│   └── Project_Setup_Manual.md          # This manual
│
├── evidence/                             # Screenshot evidence per scenario
│   ├── app_login_admin/
│   │   └── 2026-04-21/
│   │       ├── 001-login-page.png
│   │       ├── 002-mobile-entered.png
│   │       └── ...
│   ├── register_successful_registration/
│   │   └── 2026-04-21/
│   │       └── ...
│   └── <scenario_name>/
│       └── <date>/
│           └── NNN-description.png
│
├── bug_reports/                          # Generated bug reports
│   └── <scenario_name>/
│       └── <bug-title>.md
│
└── .claude_skills/                       # Agent skill definitions
    └── playwright-cli/
        └── SKILL.md
```

Naming Conventions

Item	Convention	Example
Feature files	`snake_case.feature`	`app_login.feature`

Evidence folders	`<scenario_tag_name>/`	`register_empty_mobile/`
Evidence subfolders	`YYYY-MM-DD/`	`2026-04-21/`
Screenshots	`NNN-description.png`	`001-login-page.png`
Bug reports	`kebab-case.md`	`registration-silent-login.md`
Fixtures	`kebab-case.js` or `.yml`	`check-response.js`

---

5. Configuration Files

`fixtures/credentials.yml`

This is the **single source of truth** for all environment URLs and test credentials.

```yaml
app:
  url: "https://app.development.qawafel.dev/home"
  mobile: "538989808"
  otp: "201111"

  Vendor_Store:
  mobile: "538989802"
  otp: "201111"
```

Rules:

- Never hardcode credentials in feature files or prompts
- The execution prompt reads this file automatically
- Mask passwords in bug reports

---

6. Writing Feature Files

Structure

Every `.feature` file follows this structure:

```
Feature: <Feature Name>
  As a <role>
  I want to <action>
  So that <benefit>

  Background:
    Given <shared preconditions>

  # ──────────────── Positive Scenarios ────────────────

  @tag1 @positive @smoke
  Scenario: <Descriptive name>
    When <action>
    Then <expected result>
```

```
# ──────────────── Negative Scenarios ────────────────

@tag1 @negative
Scenario: <Descriptive name>
  When <invalid action>
  Then <error handling>

# ──────────────── Failed Registration Scenarios ────────────────

@tag1 @failed
Scenario: <Descriptive name>
  Given <precondition>
  When <action that should fail>
  Then <failure behavior>
```

Smart BDD Rules

1. **No click-by-click steps** — Use business-level language
2. **Unique test data per scenario** — Each scenario gets its own mobile number
3. **Group by category** — Positive, Negative, Failed sections with comment headers
4. **Tags** — `@feature @type @priority` (e.g., `@register @negative`)
5. **Arabic UI labels** — Quote exact UI text in double quotes
6. **Background** — Shared preconditions only, applied to ALL scenarios in the file

Example: Complete Feature File

See `scenarios/Admin/register.feature` for a full reference with:

- 2 positive scenarios
- 3 negative scenarios
- 3 failed registration scenarios

---

7. Prompt Templates

The project uses **3 core prompts** stored in `prompts/`:

Prompt	File	Purpose
Execution	`Execution.md`	Execute a Gherkin scenario using playwright-cli
Bug Report	`Bug_Report.md`	Generate structured bug report from execution failures
GitHub Issue	`Github_issue.md`	Publish bug report as GitHub issue via `gh` CLI

Usage Flow

```
1. Paste Execution.md prompt → provide scenario file path
2. Agent executes scenario, captures evidence
3. If failed → paste Bug_Report.md prompt → agent generates report
4. To publish → paste Github_issue.md prompt → provide repo + report path
```

---

8. Execution Workflow

Step 1: Start a new agent session

Open VS Code Copilot Chat and paste the contents of `prompts/Execution.md`.

Step 2: Provide the scenario

```
Execute the scenario in: scenarios/Admin/app_login.feature
```

Step 3: Agent executes

The agent will:

1. Read `fixtures/credentials.yml` for URLs and credentials
2. Open a headed Chromium browser via `playwright-cli`
3. Follow the scenario steps
4. Capture screenshots at each meaningful step
5. Store evidence in `evidence/<scenario_name>/<date>/`
6. Provide a summary at the end

Step 4: Review results

Check the evidence folder for screenshots and the agent's summary for pass/fail status.

---

9. Evidence Collection

Evidence is stored automatically during execution:

```
evidence/
└── <scenario_name>/
    └── YYYY-MM-DD/
        ├── 001-login-page.png
        ├── 002-mobile-entered.png
        ├── 003-otp-screen.png
        └── ...
```

Rules:

- Three-digit numeric prefix for ordering
- Short descriptive suffix
- One folder per scenario execution per date
- Evidence preserved even on failure (captures failure state)

---

10. Bug Reporting

When a scenario fails, use the bug report prompt to generate a structured report.

Steps:

1. Paste `prompts/Bug_Report.md` into the agent session
2. The agent reads the execution summary, evidence, and original scenario

3. A structured bug report is generated in `bug_reports/<scenario_name>/`

Bug Report Structure:

- Title, Environment, Severity
- Preconditions, Steps to Reproduce
- Expected vs Actual Result
- Evidence references, Console Errors
- Additional Notes

---

11. GitHub Issue Publishing

Steps:

1. Paste `prompts/Github_issue.md` into the agent session
2. Provide: repository name, issue title, and bug report file path
3. The agent runs `gh issue create --body-file <path>`
4. Returns the created issue URL

Example:

```
gh issue create --repo "qawafel/quality-onboarding" \
  --title "[Registration] Silent login on existing mobile" \
  --body-file "bug_reports/register_already_registered_mobile/registration-silent-login-on-existing-mobile.md"
```

---

12. Fixtures & Helper Scripts

`fixtures/upload-files.js`

Uploads a test file to all `<input type="file">` elements on the page:

```
async (page) => {
  const inputs = await page.locator('input[type=file]').all();
  const filePath = 'fixtures/uploads/commercial-registration.jpg';
  for (let i = 0; i < inputs.length; i++) {
    await inputs[i].setInputFiles(filePath);
  }
  return 'uploaded to ' + inputs.length + ' inputs';
}
```

`fixtures/check-response.js`

Intercepts API responses during form submission for debugging:

```
async (page) => {
  let responses = {};
  const handler = async (response) => {
    const url = response.url();
    if (url.includes('customers/register') || url.includes('customers/addresses')) {
```

```
      try { responses[url.split('/').pop()] = await response.json(); }
      catch(e) { responses['error'] = e.message; }
    }
  };
  page.on('response', handler);
  // Click Create Account button and wait for response
  // ...
}
```

`fixtures/expire-otp.js`

Overrides `Date.now` to simulate OTP timer expiry (+10 minutes):

```
async (page) => {
  await page.evaluate(() => {
    const realDateNow = Date.now.bind(Date);
    const offset = 10 * 60 * 1000;
    Date.now = () => realDateNow() + offset;
  });
}
```

Adding New Fixtures

1. Create a `.js` file in `fixtures/` exporting an `async (page) => {}` function
2. Use via `playwright-cli run-code fixtures/<name>.js` during execution
3. Store any required test files in `fixtures/uploads/`

---

13. Complete Prompt Reference

Below are all prompts used in this project, including **4 additional prompts** for missing/required workflows.

Prompt 1: Scenario Generation (Feature File Authoring)

```
# Scenario Generation Prompt

You are helping me create Gherkin BDD scenarios for a Qawafel application feature.

Before doing anything else:

1. Read `fixtures/credentials.yml` for available environment URLs, mobile numbers, and OTP
values.
2. If a feature file already exists at the path I provide, read it first to understand
existing scenarios.

I will give you:
- The feature name and description
- The application area (Admin app or Vendor Store)
- The user role
- The positive flows to cover
- The negative/edge cases to cover
```

```
Follow these rules strictly:

1. Use **smart BDD style** — business-level steps, NOT click-by-click UI actions.
2. Group scenarios by category with comment headers:
   - `# ──────────────── Positive Scenarios ────────────────`
   - `# ──────────────── Negative Scenarios ────────────────`
   - `# ──────────────── Failed Scenarios ────────────────`
3. Use **unique test data per scenario** — each scenario must have its own mobile number.
4. Use tags: `@feature @type @priority` (e.g., `@login @negative`).
5. Quote Arabic UI labels exactly in double quotes.
6. Use Background for shared preconditions applied to ALL scenarios.
7. Include the Feature header with As a / I want / So that.
8. Limit negative scenarios to **3 max** — pick the most impactful cases.
9. Every scenario name must be **descriptive and unique** within the file.
10. Do not add step definitions or automation code — these are for manual execution.

Output format: A complete `.feature` file ready to save.
```

Prompt 2: Scenario Execution (Already exists in `prompts/Execution.md`)

(See Section 8 above for details)

Prompt 3: Bug Report Generation (Already exists in `prompts/Bug_Report.md`)

(See Section 10 above for details)

Prompt 4: GitHub Issue Publishing (Already exists in `prompts/Github_issue.md`)

(See Section 11 above for details)

Prompt 5: Feature File Grooming

```
# Feature File Grooming Prompt

You are helping me groom and refine an existing Gherkin feature file.

Before doing anything else:

1. Read the feature file I provide.
2. Read `fixtures/credentials.yml` for available test data.
3. Review any execution evidence in `evidence/` related to this feature.

I want you to review and improve the feature file following these rules:

1. Ensure all scenarios use **unique mobile numbers** — no two scenarios share the same test
data.
2. Group scenarios by category with comment section headers (Positive, Negative, Failed).
3. Apply proper tags: `@feature @type` on each scenario.
4. Remove redundant or overlapping negative scenarios — keep max 3 of the most impactful.
```

```
5. Ensure steps use **business-level language**, not click-by-click instructions.
6. Verify the Background section contains only truly shared preconditions.
7. Check that all Arabic UI labels are quoted correctly.
8. Ensure scenario names are descriptive and unique.
9. Add any missing edge cases you identify (max 2 suggestions).
10. Present the changes as a diff summary, then provide the complete updated file.

Do NOT:
- Change passing scenario logic
- Add automation code
- Remove scenarios without explaining why
```

Prompt 6: Test Data Management

```
# Test Data Management Prompt

You are helping me manage test data for the Qawafel BDD test project.

Before doing anything else:

1. Read `fixtures/credentials.yml` for current test data.
2. Read all `.feature` files in `scenarios/` to find all mobile numbers and test data in
use.

Produce:

1. **Test Data Registry** — A table listing every mobile number used, which scenario uses
it, and its purpose:

   | Mobile | Feature File | Scenario | Purpose |
   |--------|-------------|----------|---------|
   | 538989808 | app_login.feature | Admin login | Admin account |
   | ... | ... | ... | ... |

2. **Conflict Check** — Flag any duplicate mobile numbers used across different scenarios.

3. **Available Range** — Suggest the next available mobile numbers following the project's
numbering pattern.

4. **Credentials File Update** — If any new credentials were used during execution that are
not in `credentials.yml`, suggest updates.

Rules:
- Scan ALL feature files recursively
- Include credentials from both `credentials.yml` and inline scenario data
- Flag any hardcoded credentials that should be moved to `credentials.yml`
```

Prompt 7: Execution Summary & Report

```
# Execution Summary Prompt

You are helping me generate a consolidated execution summary for a test run.

Before doing anything else:

1. Read all feature files in `scenarios/`.
2. Scan the `evidence/` folder to determine which scenarios have been executed.
3. Read any bug reports in `bug_reports/`.

Produce a **Test Execution Report** with:

### 1. Execution Summary Table

| # | Feature | Scenario | Status | Evidence | Bug Report |
|---|---------|----------|--------|----------|------------|
| 1 | Login | Successful admin login | ✅ PASS | evidence/app_login_admin/ | — |
| 2 | Register | Empty mobile | ✅ PASS | evidence/register_empty_mobile/ | — |
| 3 | Register | Complete form | ❌ FAIL | evidence/register_complete_form/ |
bug_reports/... |

### 2. Statistics
- Total scenarios: X
- Passed: X
- Failed: X
- Not executed: X
- Pass rate: X%

### 3. Defects Found
- List each bug report with title, severity, and GitHub issue link (if published)

### 4. Recommendations
- Scenarios that should be re-tested
- Missing test coverage areas
- Test data issues found

Rules:
- Base status on evidence existence and bug reports — do not guess
- List unexecuted scenarios separately
- Include evidence folder paths as references
```

Prompt 8: Project Setup (New Project Bootstrap)

```
# Project Setup Prompt

You are helping me set up a new Qawafel BDD test project from scratch.

Create the following project structure:

1. Initialize with `npm init -y`
```

```
2. Create these folders:
   - `scenarios/Admin/` — for Gherkin feature files
   - `fixtures/` — for credentials and helper scripts
   - `fixtures/uploads/` — for test file uploads
   - `prompts/` — for reusable agent prompts
   - `evidence/` — for screenshot evidence (empty, populated during execution)
   - `bug_reports/` — for generated bug reports (empty, populated on failures)

3. Create `fixtures/credentials.yml` with this template:
   ```yaml
   app:
     url: "<APP_URL>"
     mobile: "<ADMIN_MOBILE>"
     otp: "<OTP_CODE>"

     Vendor_Store:
       mobile: "<VENDOR_MOBILE>"
       otp: "<OTP_CODE>"
```

4. Copy all prompt templates into `prompts/` :
   - Execution.md
   - Bug_Report.md
   - Github_issue.md
5. Verify `playwright-cli` is available and install browsers:

   ```
   playwright-cli install-browser
   ```

6. Authenticate GitHub CLI:

   ```
   gh auth login --hostname github.com --git-protocol https --web
   ```

7. Create a sample feature file in `scenarios/Admin/` with one positive and one negative scenario as a template.

After setup, print:

- The full folder tree
- Confirmation of each step
- Any missing prerequisites

Do NOT:

- Install unnecessary npm packages
- Create automation framework code (this is a manual execution project)
- Modify VS Code settings

```
---

## Quick Reference Card
```
```

```
| Action | Command / Prompt |
|--------|-----------------|
| Create project | `npm init -y` + folder structure |
| Write scenarios | Use Prompt 1 (Scenario Generation) |
| Execute scenarios | Use Prompt 2 (Execution.md) |
| Report bugs | Use Prompt 3 (Bug_Report.md) |
| Publish to GitHub | Use Prompt 4 (Github_issue.md) |
| Groom feature files | Use Prompt 5 (Feature File Grooming) |
| Manage test data | Use Prompt 6 (Test Data Management) |
| Generate summary | Use Prompt 7 (Execution Summary) |
| Bootstrap new project | Use Prompt 8 (Project Setup) |

---

*End of User Manual*
```